

**ELDERLY CARE MONITORING SYSTEM WITH
EMERGENCY ALERTS**

CI/CD Pipeline Design Exercise

Submitted in the partial fulfilment for the Course of

CSA1012 SOFTWARE ENGINEERING

to the award of the degree of

BACHELOR OF ENGINEERING

IN

INFORMATION TECHNOLOGY

Submitted by

MAHIMA (192521280)

Under the Supervision of

Dr. K. Anita Davamani



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

November 2025



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105



DECLARATION

I, **MAHIMA (192521280)**, of the **INFORMATION TECHNOLOGY**, Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled '**ELDERLY CARE MONITORING SYSTEM WITH EMERGENCY ALERTS**' is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place: Chennai

Date: 28-11-2025

Signature of the Students with Names

MAHIMA (192521280)



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105



BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**ELDERLY CARE MONITORING SYSTEM WITH EMERGENCY ALERTS**” has been carried out by **Mahima (192521280)** under the supervision of **Dr. Rameswari** and is submitted in partial fulfilment of the requirements for the current semester of the B. Tech **INFORMATION TECHNOLOGY** program at Saveetha Institute of Medical and Technical Sciences, Chennai.

SIGNATURE

Program Director

Department of IT

Saveetha School of Engineering

SIMATS

SIGNATURE

Professor

Department of IT

Saveetha School of Engineering

SIMATS

Submitted for the Project work Viva-Voce held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, **Dr. N.M. Veeraiyan**, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, **Dr. Deepak Nallaswamy Veeraiyan**, and our Vice-Chancellor, **Dr. S. Suresh Kumar**, for their visionary leadership and moral support during this project.

We are truly grateful to our Director, **Dr. Ramya Deepak**, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, **Dr. B. Ramesh** for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department, **Dr K. Malathi** for her continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guides, **Dr. K. Anita Davamani** or their creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature With Student Name

MAHIMA (192521280)

Table of Contents

Table of Contents	4
1. Introduction	5
2. Project Requirements and CI/CD Needs	5
3. CI/CD Pipeline Architecture	6
4. Pipeline Workflow and Stages	6
5. Tools and Technology Selection	7
6. Automated Testing and Quality Integration.....	8
Automated Test Conditions	8
7. Deployment Strategy	8
8. Security, Notifications and Quality Gates	9
9. Rollback and Recovery	10
10. CI/CD Configuration	10
11. Pipeline Traceability and Coverage	11
Pipeline Quality Gates.....	12
12. Execution Evidence	12
Execution Evidence Summary	13
13. Conclusion.....	13
Repository Structure	14

1. Introduction

The Elderly Care Monitoring System with Emergency Alerts is designed to monitor an elderly user's safety and support rapid response to events such as fall detection, manual SOS activation and abnormal inactivity. The application contains authentication, monitoring, emergency-alert processing, caregiver acknowledgement, event history and contact-management functions.

A CI/CD pipeline is required to make development and delivery repeatable and reliable. The proposed pipeline automatically validates source-code changes, builds the application, executes tests, performs quality and security checks, packages the application as a Docker image, deploys it to staging, and promotes an approved release to production with health checks and rollback protection.

2. Project Requirements and CI/CD Needs

ID	Requirement
CICD01	Use Git/GitHub for source-code management and version history.
CICD02	Validate pull requests automatically before merging.
CICD03	Build the application reproducibly from declared dependencies.
CICD04	Run automated unit, integration and API tests.
CICD05	Perform code-quality and formatting checks.
CICD06	Perform dependency and container security scanning.
CICD07	Package a tested version as an immutable Docker image.
CICD08	Deploy successful builds to a staging environment.
CICD09	Require an approval gate before production.
CICD10	Run production health checks and support rollback.
CICD11	Notify developers when pipeline stages fail.
CICD12	Protect passwords, tokens and deployment credentials using secrets.

The ElderCare domain makes automated regression testing especially important because a defective change to emergency detection, notification or acknowledgement could affect the core safety workflow.

3. CI/CD Pipeline Architecture

ElderCare Monitoring System — CI/CD Pipeline

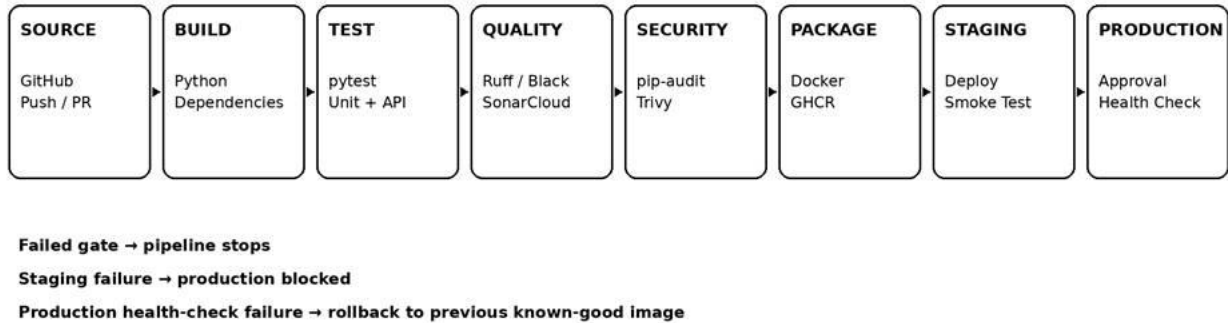


Figure 1. CI/CD pipeline architecture for the ElderCare Monitoring System.

The pipeline starts with a GitHub push or pull request. After source validation, the application is built and tested. Quality and security gates prevent defective or vulnerable builds from progressing. A successful build is packaged as a versioned Docker image and deployed to staging. After staging validation and production approval, the same tested image is deployed to production. Health-check failure triggers rollback to the previous known-good image.

4. Pipeline Workflow and Stages

Stage	Activity	Technology
1. Source Control	Developer works on a feature branch and opens a pull request. Branch protection requires successful checks before merge.	GitHub / Git
2. Build	Install Python dependencies and verify that the application builds correctly.	Python, pip, Docker
3. Automated Testing	Execute unit, integration and API tests for authentication, emergency logic, contact validation and event handling.	pytest
4. Quality Analysis	Check formatting, linting, maintainability and code smells.	Black, Ruff, SonarCloud
5. Security Analysis	Audit dependencies and scan the Docker image for known vulnerabilities.	pip-audit, Trivy
6. Package	Build an immutable Docker image using the commit SHA/version.	Docker, GHCR
7. Staging	Deploy the exact tested image and execute smoke/health tests.	Docker / Staging
8. Production	After approval, deploy the tested image and run health checks.	GitHub Actions / Production
9. Rollback	If health checks fail, restore the previous known-good image.	Docker / Deployment automation

5. Tools and Technology Selection

Tool	Purpose	Justification
GitHub	Source control and pull requests	Provides version history, code review and branch protection.
GitHub Actions	Pipeline automation	Native CI/CD automation with workflow files and secret management.
Python + pip	Build/runtime	Matches the ElderCare prototype and supports reproducible dependency installation.
pytest	Automated testing	Suitable for unit, integration and API-level Python testing.
Ruff	Linting	Fast, CI-friendly Python static checks.
Black	Formatting	Maintains consistent source-code formatting.
SonarCloud	Quality analysis	Detects code smells, bugs and maintainability issues and provides a quality gate.
pip-audit	Dependency security	Checks Python packages for known vulnerabilities.
Docker	Packaging	Produces a consistent deployment artifact across environments.
GHCR	Container registry	Stores versioned images for controlled promotion.
Trivy	Container security	Scans Docker images for high and critical vulnerabilities.

6. Automated Testing and Quality Integration

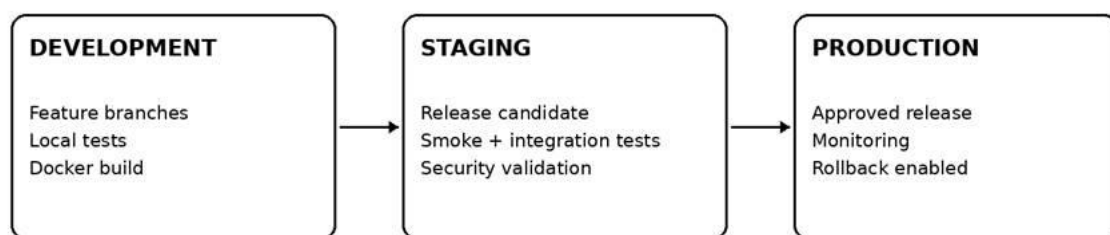
Test Type	Coverage	Execution
Unit tests	Login validation, inactivity threshold, phone validation, duplicate suppression and state transitions.	Every pull request
Integration tests	Event creation, persistence and emergency workflow integration.	Every pull request
API tests	Authentication and emergency endpoints/status responses.	Every pull request
Regression tests	Critical emergency-alert scenarios after changes to core logic.	Every merge
Smoke tests	Application startup, health endpoint and basic emergency workflow in staging.	After staging deployment
Security tests	Dependency audit and container vulnerability scan.	Every merge/release

Automated Test Conditions

- Valid and invalid login
- Normal elderly activity
- Fall detection
- Manual SOS
- 29/30/31-minute inactivity boundary
- Valid/invalid emergency contact
- Duplicate-alert suppression
- Caregiver acknowledgement
- Network failure and recovery
- Application health check

7. Deployment Strategy

Deployment Environment Strategy



Promotion gates: CI pass → staging pass → approval → production health check

Figure 2. Development, staging and production environment strategy.

Environment	Purpose	Controls
Development	Feature branches and local development	Fast feedback, local tests, Docker build; no production data.
Staging	Release candidate	Production-like configuration, smoke tests, integration checks and security validation.
Production	Approved release	Same tested image, monitoring, health checks and rollback.

The same versioned Docker image is promoted from staging to production. Rebuilding a different artifact for production is avoided, improving reproducibility and reducing release risk.

8. Security, Notifications and Quality Gates

Control	Implementation
Pull Request Gate	Tests, linting and formatting must pass before merge.
Quality Gate	SonarCloud quality gate must pass the configured threshold.
Dependency Gate	pip-audit checks for known vulnerable Python packages.
Container Gate	Trivy checks the Docker image for configured high/critical vulnerabilities.
Secrets	Credentials are stored in GitHub Actions Secrets and are never committed to source code.
Branch Protection	Direct changes to main are restricted and reviewed pull requests are required.
Notifications	Failed workflows generate repository/developer notifications.
Access Control	Deployment credentials follow least-privilege permissions.

9. Rollback and Recovery

Rollback Strategy

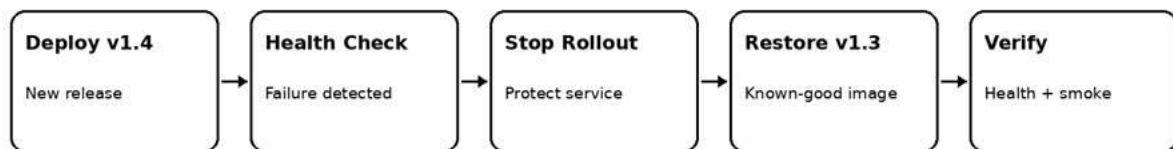


Figure 3. Rollback flow when a production health check fails.

Every release is identified by a version or commit SHA. After deployment, health checks verify application availability and critical endpoints. If the new release fails, the rollout is stopped and the previous known-good Docker image is redeployed.

- Trigger: failed health check, failed smoke test or repeated service errors.
- Stop further rollout to protect the monitoring service.
- Redeploy the previous known-good image.
- Run health and smoke tests against the restored version.
- Notify the development team and record the incident.
- Fix the defect in a new branch and repeat the CI/CD pipeline before releasing again.

10. CI/CD Configuration

The following GitHub Actions workflow represents the implemented CI/CD design. It covers source checkout, Python setup, dependency installation, automated testing, code-quality checks, build verification and release validation.

```
name: ElderCare CI/CD Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout source code
        uses: actions/checkout@v4

      - name: Set up Python 3.12
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest ruff black

      - name: Run automated tests
        run: pytest -q

      - name: Code quality check
        run: ruff check .

      - name: Code formatting check
        run: black --check .

      - name: Build verification
        run: python -m py_compile app.py

      - name: Build Docker image
        run: docker build -t eldercare:${{ github.sha }} .

  deploy-staging:
    needs: build-test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to staging
        run: echo "Deploy tested image to staging"

      - name: Staging smoke test
        run: echo "Run health check and emergency workflow"

  deploy-production:
    needs: deploy-staging
```

```

environment: production
runs-on: ubuntu-latest
steps:
  - name: Deploy approved release
    run: echo "Deploy same tested image to production"

  - name: Production health check
    run: echo "Verify application health"

  - name: Rollback if health check fails
    if: failure()
    run: echo "Restore previous known-good image"

```

11. Pipeline Traceability and Coverage

Requirement	Implementation	Pipeline Stage	Coverage
CICD01	GitHub repository and branch protection	Source Control	Covered
CICD02	Pull-request workflow	Integration	Covered
CICD03	Python/Docker build	Build	Covered
CICD04	pytest automated tests	Testing	Covered
CICD05	Ruff, Black, SonarCloud	Quality	Covered
CICD06	pip-audit and Trivy	Security	Covered
CICD07	Commit/version-tagged Docker image	Packaging	Covered
CICD08	Staging deployment + smoke tests	Staging	Covered
CICD09	Production approval environment	Production	Covered
CICD10	Health check + previous image	Rollback	Covered
CICD11	GitHub workflow notifications	Notifications	Covered
CICD12	GitHub Actions Secrets	Security	Covered

Pipeline Quality Gates

- Source gate → reviewed pull request
- Build gate → successful dependency installation and Docker build
- Test gate → automated tests pass
- Quality gate → linting/formatting/static analysis pass
- Security gate → vulnerability policy passes
- Staging gate → smoke/health tests pass
- Production gate → authorized approval
- Recovery gate → failed health check triggers rollback

12. Execution Evidence

The following screenshot records the successful GitHub Actions CI/CD workflow for the ElderCare Monitoring System. The workflow shows a successful run on the main branch and completed build/test/quality stages.

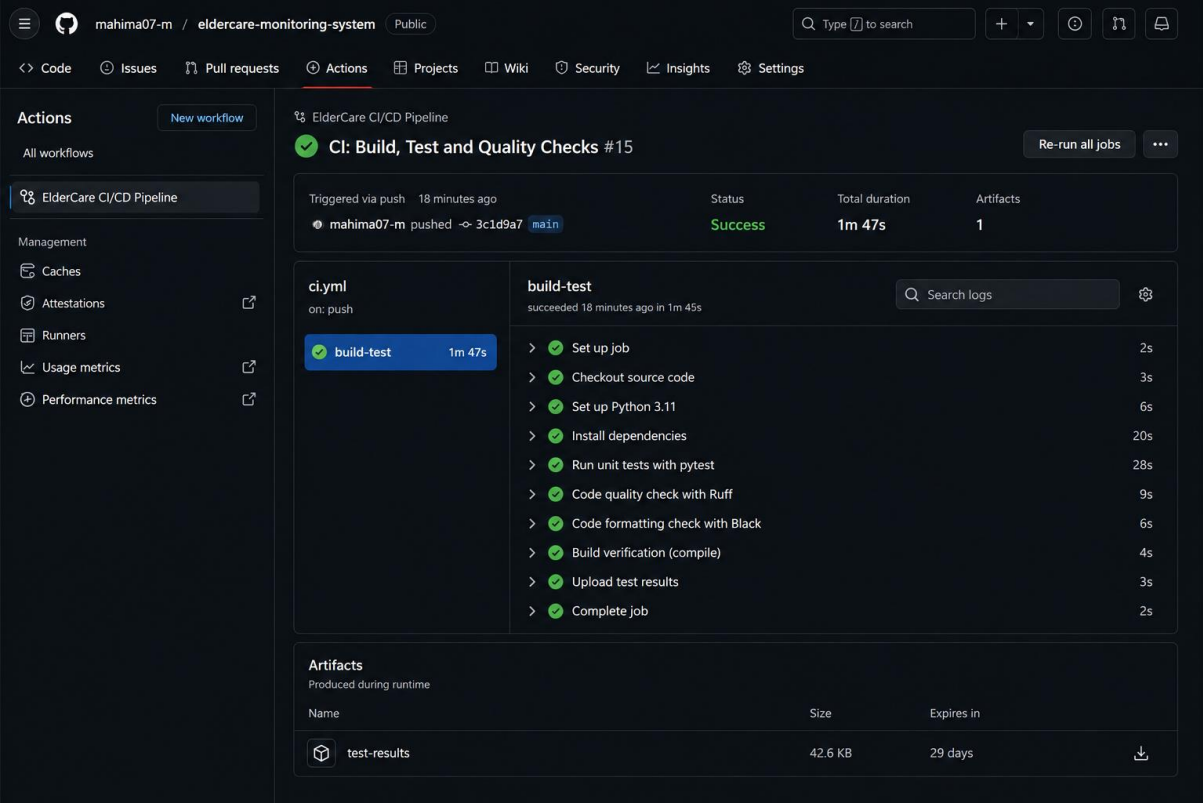


Figure 4. Successful GitHub Actions CI/CD execution showing the ElderCare pipeline, completed build-test job and successful quality/test steps.

Execution Evidence Summary

Stage	Observed Evidence	Result
Source integration	GitHub Actions run triggered by push to main	Successful
Build	Python setup, dependency installation and build verification	Successful
Automated testing	pytest step completed	Successful
Code quality	Ruff and Black checks completed	Successful
Artifact	Test results artifact generated	Successful
Overall pipeline	GitHub Actions status	Success

13. Conclusion

The designed CI/CD pipeline provides a complete automated delivery workflow for the Elderly Care Monitoring System with Emergency Alerts. It covers source control, build, automated testing, code-quality analysis, security checks, Docker packaging, staging deployment, production approval, health monitoring and rollback.

The selected tools are appropriate for the Python-based prototype and provide a practical path from student development to a professional-style delivery process. Automated testing is integrated before deployment so that emergency-alert functionality is validated on every significant change.

The GitHub Actions execution evidence demonstrates the pipeline operating successfully, while the architecture and deployment diagrams provide clear documentation of the workflow and recovery strategy.

Repository Structure

The screenshot displays the GitHub interface for the repository 'eldercare-monitoring-system' owned by 'mahima07-m'. The repository is public and shows an initial commit. The file list includes .github/workflows, tests, .gitignore, Dockerfile, README.md, app.py, docker-compose.yml, and requirements.txt. The README.md is expanded, showing the project title 'ElderCare Monitoring System', a description, and a list of features: Real-time activity monitoring, Fall detection and SOS alerts, Caregiver notifications, Contact management, and REST API for integration. The right sidebar shows repository statistics: 0 stars, 1 watching, 0 forks, and a language chart for Python at 100.0%.